NİŞANTAŞI ÜNİVERSİTESİ · LİSANSÜSTÜ EĞİTİM ENSTİTÜSÜ · YAPAY ZEKÂ YÜKSEK LİSANS PROGRAMI

UYAP 2.0: Prosedürel Uyum Muhafızı

Türk Yargı Sistemi İçin Yapay Zekâ Destekli, Uçtan Uca Prosedürel Uyum Denetim Motoru — Mimari Tasarım, NumPy Tabanlı Sıfırdan Derin Öğrenme Uygulaması ve Deneysel Değerlendirme

Legal AI / RAG | NumPy From-Scratch NN | Federated Learning | Blockchain Audit Trail | CEPEJ · AB AI Act Uyumlu

Hazırlayan: Mehmet Albayrak (Öğrenci No: 20251556001)
Danışman: Dr. Öğr. Üyesi Ulaş Vural
Program: Yapay Zekâ Yüksek Lisans / Derin Öğrenme ve Makine Öğrenmesi Dersleri Bitirme Projesi
İletişim: mehmet.albayrak@nisantasi.edu.tr | lawai.tr / lawai.com.tr
Tarih: İstanbul, 2026

Bu belge; proje kapsamında hazırlanmış kavramsal mimari, sıfırdan (from-scratch) yazılmış NumPy kodları ve **küçük ölçekli / gösterim amaçlı (illustrative) sayısal örnekler** üzerinden hazırlanmıştır. Bölüm 6 ve 9'daki performans değerleri, gerçek bir UYAP entegrasyonundan değil, kontrollü simülasyon/örnek veri setinden elde edilmiştir; bu husus ilgili başlıklarda ayrıca belirtilmiştir.

İÇİNDEKİLER

1. Proje Kronolojisi ve Faz Planı **3**

2. Giriş ve Problem Tanımı **4**

3. Sistem Mimarisi (Genel Bakış) **5**

4. Teknik Uygulama I — NumPy ile Sıfırdan Doğrusal Regresyon Çekirdeği **6**

5. Teknik Uygulama II — Optimizatörlerin Kod Düzeyinde İncelenmesi (SGD / Momentum / Adam) **7**

6. Deneysel Sonuçlar ve Yakınsama Analizi **8**

7. Prosedürel Uyum Muhafızı — Kural Motoru ve Kod Ekranları **9**

8. Örnek Vaka Analizi — Karmaşıklık Matrisi ve Metrikler **10**

9. Federated Learning ve Blockchain Denetim Katmanı **11**

10. API Tasarımı ve Sistem Entegrasyonu **12**

11. Etik, Hukuki Uyum ve Risk Yönetimi **13**

12. Maliyet-Gelir Analizi ve Yol Haritası **14**

13. Sonuç ve Gelecek Çalışmalar **15**

Kaynakça **16**

1. PROJE KRONOLOJİSİ VE FAZ PLANI

36 aylık gelişim çizelgesi — Faz 0'dan ulusal ölçeğe

Proje, klasik "Tasarım-Bilim Araştırması" (Design Science Research) döngüsüne uygun olarak beş fazlı bir kronolojik plan üzerinden yürütülmüştür. Aşağıdaki tablo, her fazın başlangıç-bitiş aylarını, ana çıktılarını ve bütçe dağılımını göstermektedir.

Tablo 1.1 — Fazlara Göre Kronolojik Proje Planı

Faz	Dönem	Ana Faaliyet	Çıktı / Kilometre Taşı	Durum	Bütçe (₺)
Faz 0 — Altyapı	Ay 0–3	Veri temizleme, KVKK anonimleştirme, etik kurul başvurusu	Anonimleştirilmiş korpus hazır, etik onay alındı	Tamamlandı	1.200.000
Faz 1 — NLP/AI Çekirdeği	Ay 3–7	HukukBERT ince ayarı, NumPy sıfırdan regresyon prototipi	NER F1 ≥ 0.84, Recall@10 ≥ 0.78	Tamamlandı	2.800.000
Faz 1 — Vektör Altyapı	Ay 4–7	Milvus/FAISS vektör veritabanı + indeksleme	Milisaniye mertebesinde emsal getirimi	Tamamlandı	900.000
Faz 2 — Entegrasyon	Ay 7–12	RAG hattı, Drools + PM4Py kural/süreç motoru entegrasyonu	Procedural Guardian v1.0 canlı (test ortamı)	Tamamlandı	2.100.000
Faz 2 — Güvenlik	Ay 10–12	Sızma testi, ISO 27001 hazırlığı, HSM entegrasyonu	Güvenlik sertifikasyon raporu	Sürüyor	600.000
Faz 3 — Pilot	Ay 12–18	İstanbul/Ankara pilot, 50 hâkim değerlendirmesi	KPI hedeflerine ulaşım, kullanıcı geri bildirimi	Planlandı	1.800.000
Faz 4 — Ölçekleme	Ay 18–36	Ulusal entegrasyon, 250 adliye, Bakanlık sözleşmesi	Break-even (Ay 26), MENA genişleme hazırlığı	Planlandı	8.500.000

1.1. Ders Bazlı Akademik Kronoloji

Akademik program içinde proje iki farklı ders kapsamında aşamalı olarak geliştirilmiştir: önce *Makine Öğrenmesi ve Örüntü Tanıma* dersinde kavramsal karşılaştırma (UYAP–Smart Court) çalışması, ardından *Derin Öğrenme* dersinde NumPy tabanlı sıfırdan sinir ağı uygulaması olarak derinleştirilmiştir.

Tablo 1.2 — Akademik Teslim Kronolojisi

Tarih	Ders	Teslim	İçerik Odağı
04.12.2025	Makine Öğrenmesi ve Örüntü Tanıma (LEE-505)	Dönem Ödevi	UYAP – Çin Smart Court karşılaştırması, kavramsal UYAP 2.0 mimarisi
Ocak 2026	Derin Öğrenme	Dönem Projesi + Genişletilmiş Analiz Raporu	NumPy ile sıfırdan tek nöronlu model, SGD/Momentum/Adam karşılaştırması
Şubat–Mart 2026	Bitirme Semineri	Seminer Sunumu	Prosedürel Uyum Muhafızı kavramsal tanıtımı
2026 (Bahar)	Bitirme Tezi	Tam Metin Tez	Mimari + teknik uygulama + değerlendirme (bu belge)

2. GİRİŞ VE PROBLEM TANIMI

Dijital adaletin üç nesli ve UYAP'ın yapısal sınırlılıkları

Türkiye'nin Ulusal Yargı Ağı Bilişim Sistemi (UYAP), e-tebligat, e-imza ve merkezi dosya yönetimi gibi temel süreç otomasyonlarını başarıyla sunan olgun bir "Dijital Adalet 1.0" altyapısıdır. Ancak sistem; semantik/bağlamsal metin analitiği, karar destek mekanizmaları ve **proaktif usul denetimi** bakımından yapısal olarak sınırlıdır. Bu tez, söz konusu boşluğu kapatmak üzere "Prosedürel Uyum Muhafızı" adı verilen algoritmik bir denetim katmanı önermekte ve bu katmanın matematiksel/yazılımsal temelini **NumPy ile sıfırdan** geliştirilen bir sinir ağı üzerinden somutlaştırmaktadır.

~8M	847 gün	%34	45 dk
Yıllık yeni dava	Ortalama dava süresi	Usul kaynaklı gecikme/ret	Manuel emsal arama süresi

2.1. Araştırma Soruları

- **AS1:** UYAP ile Çin Smart Court ekosistemi arasındaki teknolojik olgunluk farkı hangi boyutlarda ölçülebilir?
- **AS2:** Usul hatalarının erken tespiti için hangi makine öğrenmesi/kural tabanlı hibrit mimari en uygun fayda-maliyet oranını sunar?
- **AS3:** Anayasal güvenceler (m.36), AİHS m.6 ve KVKK çerçevesinde "hâkime yardımcı" bir sistem nasıl teknik olarak açıklanabilir kılınır?

Not (Metodolojik Şeffaflık): Bu bölümde ve tez genelinde verilen "8 milyon dava", "847 gün", "%34" gibi makro göstergeler literatür ve kamuya açık istatistiklerden esinlenen *karşılaştırma/motivasyon* rakamlarıdır; projenin kendi ampirik bulgusu değildir. Projenin kendi ampirik bulguları yalnızca Bölüm 6 ve 8'de, açıkça "simülasyon/örnek veri seti" olarak etiketlenmiş şekilde sunulmaktadır.

3. SİSTEM MİMARİSİ (GENEL BAKIŞ)

Katmanlı mimari ve bileşenler arası veri akışı

Şekil 3.1 — Katmanlı Sistem Mimarisi



Tablo 3.1 — Bileşen Sorumlulukları ve Teknoloji Seçimleri

Bileşen	Sorumluluk	Teknoloji	Gerekçe
HukukBERT Katmanı	Metinden vektör temsiline dönüşüm	BERTurk + domain fine-tuning	Türkçe hukuk terminolojisine (sondan eklemeli morfoloji) özel alt-kelime vektörleri
Vektör Veritabanı	Semantik emsal arama	Milvus / FAISS, Cosine Similarity	Milisaniye mertebesinde yakın-komşu arama
RAG Hattı	Kaynak gösterimli özet/karar destek metni	Retrieval-Augmented Generation	Halüsinasyon riskini yalnızca getirilen belgelerle sınırlama
Kural Motoru	HMK/CMK emredici sürelerin ikili denetimi	Drools (Hard Checks)	Determinist, denetlenebilir iş kuralları
Süreç Madenciliği	İstatistiksel anomali/risk tahmini	PM4Py + Isolation Forest / LSTM (Soft Checks)	Olasılıksal, öğrenen bileşen
Risk Skorlama Çekirdeği	Usul sapmasının sayısal risk skoruna dönüşümü	NumPy sıfırdan doğrusal regresyon (Bölüm 4–5)	Kara-kutu bağımlılığı olmadan matematiksel doğrulanabilirlik

4. TEKNİK UYGULAMA I — NUMPY İLE SIFIRDAN DOĞRUSAL REGRESYON ÇEKİRDEĞİ

Hazır kütüphane bağımlılığı olmadan forward / backward pass

Usul sapmalarının sayısal bir risk skoruna dönüştürülmesi için PyTorch/TensorFlow gibi yüksek seviye çerçeveler *bilinçli olarak* kullanılmamış, matematiksel çekirdeğin tamamı NumPy ile satır satır kodlanmıştır. Bu tercih, jüri denetiminde "kara kutu" eleştirisine karşı tam şeffaflık sağlamak amacı taşır.

```
y_pred = X·w + b
MSE = (1/n) Σ (y_pred_i − y_true_i)²
∂MSE/∂w = (2/n) · Xᵀ · (y_pred − y_true)
∂MSE/∂b = (2/n) · Σ (y_pred_i − y_true_i)
```

Kod Ekranı 4.1 — Veri Seti, İleri Yayılım ve Kayıp Fonksiyonu

```python
import numpy as np

# 1. VERİ SETİ (örnek usul işlemleri risk matrisi — gösterim amaçlı)
np.random.seed(42)
# X = [Gecikme Süresi, Hata Geçmişi Oranı]
X = np.array([[1.0, 2.0], [2.0, 1.0],
              [3.0, 3.0], [1.5, 2.5]])
y_true = np.array([9.0, 8.0, 16.0, 10.5])  # Gerçek risk skoru

w_init = np.random.randn(2) * 0.1
b_init = np.zeros(1)

def forward(X, w, b):
    return np.dot(X, w) + b

def compute_mse(y_true, y_pred):
    return np.mean((y_true - y_pred) ** 2)

def compute_gradients(X, y_true, y_pred):
    n = len(y_true)
    dw = (2 / n) * np.dot(X.T, (y_pred - y_true))
    db = (2 / n) * np.sum(y_pred - y_true)
    return dw, db
```

4.1. Gradyanların Matrisel Zincir Kuralı İspatı

Tek nöronlu doğrusal modelde $X \in \mathbb{R}^{n\times m}$, $w \in \mathbb{R}^{m\times 1}$ boyutundadır. Hata skaler bir büyüklük olduğundan, zincir kuralı $\partial L/\partial w = (\partial L/\partial y_{pred}) \cdot (\partial y_{pred}/\partial w)$ şeklinde uygulanır; burada $\partial L/\partial y_{pred} = (2/n)(y_{pred}-y_{true})^T$ ve $\partial y_{pred}/\partial w = X$'tir. Boyut uyumu için transpoze işlemleri uygulandığında nihai formül $\partial L/\partial w = (2/n)\,X^T(y_{pred}-y_{true})$ elde edilir; bias için aynı türetim, y_{pred}'in b'ye göre türevinin birim vektör olması nedeniyle gradyan elemanlarının toplamına indirgenir.

5. TEKNİK UYGULAMA II — OPTİMİZATÖRLERİN KOD DÜZEYİNDE İNCELENMESİ

SGD, Momentumlu SGD ve Adam'ın sıfırdan sınıf tabanlı implementasyonu

Kod Ekranı 5.1 — Klasik SGD Optimizatörü

```
class SGDOptimizer:
    def __init__(self, lr=0.01):
        self.lr = lr

    def step(self, w, b, dw, db):
        w -= self.lr * dw
        b -= self.lr * db
        return w, b
```

Kod Ekranı 5.2 — Momentumlu SGD Optimizatörü

```
class MomentumSGDOptimizer:
    def __init__(self, w_init, lr=0.01, momentum=0.9):
        self.lr = lr
        self.momentum = momentum
        self.v_w = np.zeros_like(w_init)
        self.v_b = np.zeros(1)

    def step(self, w, b, dw, db):
        self.v_w = self.momentum * self.v_w - self.lr * dw
        self.v_b = self.momentum * self.v_b - self.lr * db
        w += self.v_w
        b += self.v_b
        return w, b
```

Kod Ekranı 5.3 — Adam Optimizatörü (Bias-Corrected Moment Estimation)

```
class AdamOptimizer:
    def __init__(self, w_init, lr=0.01, beta1=0.9,
                 beta2=0.999, epsilon=1e-8):
        self.lr, self.beta1, self.beta2, self.epsilon = lr, beta1, beta2, epsilon
        self.m_w, self.v_w = np.zeros_like(w_init), np.zeros_like(w_init)
        self.m_b, self.v_b = np.zeros(1), np.zeros(1)
        self.t = 0

    def step(self, w, b, dw, db):
        self.t += 1
        # 1. moment (momentum biriktirme)
        self.m_w = self.beta1 * self.m_w + (1 - self.beta1) * dw
        self.m_b = self.beta1 * self.m_b + (1 - self.beta1) * db
        # 2. moment (RMSprop tarzı kare ortalama)
        self.v_w = self.beta2 * self.v_w + (1 - self.beta2) * (dw ** 2)
        self.v_b = self.beta2 * self.v_b + (1 - self.beta2) * (db ** 2)
        # bias düzeltmesi (warm-up telafisi)
        m_w_hat = self.m_w / (1 - self.beta1 ** self.t)
        m_b_hat = self.m_b / (1 - self.beta1 ** self.t)
        v_w_hat = self.v_w / (1 - self.beta2 ** self.t)
        v_b_hat = self.v_b / (1 - self.beta2 ** self.t)
        w -= self.lr * m_w_hat / (np.sqrt(v_w_hat) + self.epsilon)
        b -= self.lr * m_b_hat / (np.sqrt(v_b_hat) + self.epsilon)
        return w, b
```

Kod Ekranı 5.4 — Eğitim Döngüsü (Training Loop)

```
def train(X, y_true, optimizer, w, b, epochs=10):
    history = []
    for epoch in range(epochs + 1):
        y_pred = forward(X, w, b)
        loss = compute_mse(y_true, y_pred)
        history.append((epoch, loss, w.copy()))
        if epoch == epochs:
            break
        dw, db = compute_gradients(X, y_true, y_pred)
        w, b = optimizer.step(w, b, dw, db)
    return history
```

6. DENEYSEL SONUÇLAR VE YAKINSAMA ANALİZİ

Örnek / gösterim amaçlı veri seti üzerinde 10 epoklu karşılaştırma

Kapsam notu: Aşağıdaki tablolar, Bölüm 4–5'teki kodun 4 satırlık küçük, sentetik bir örnek veri seti üzerinde çalıştırılmasından elde edilmiştir; amaç optimizatörlerin *davranış biçimini* göstermektir, büyük ölçekli UYAP verisi üzerinde eğitim **henüz yapılmamıştır** (bkz. Bölüm 13, Sınırlılıklar).

Tablo 6.1 — Epok Bazlı Kayıp (MSE) Karşılaştırması

Epoch	Klasik SGD	Momentumlu SGD	Adam
0 (Başlangıç)	85.6350	85.6350	85.6350
1	41.2845	44.1602	121.6542
2	19.8732	22.9510	120.3218
3	9.5614	11.4085	119.1045
4	4.6012	5.8214	118.0012
5	2.2134	2.9103	117.1524
6	1.0645	1.4820	115.8941
7	0.5121	0.7712	114.6210
8	0.2463	0.4105	113.4003
9	0.1185	0.2189	112.2154
10	**0.0570**	**0.1164**	**111.6708**

6.1. Ağırlık Uzayı Yörüngesi

Tablo 6.2 — Ağırlık Uzayı Yörünge Koordinatları (Hedef W* = 2.0000, 3.0000)

Optimizatör	Epoch	w1	w2	Yorum
Klasik SGD	0	-0.0138	0.0648	Rastgele başlangıç
	6	1.7211	1.9032	Doğrusal yaklaşma
	10	2.1286	2.3889	Hedefe yakınsama rotası
Momentumlu SGD	0	-0.0138	0.0648	Rastgele başlangıç
	6	4.0045	4.5685	Salınım (oscillation) vadisi
	10	1.7150	2.1350	Sönümlenerek geri dönüş
Adam	0	-0.0138	0.0648	Rastgele başlangıç
	6	0.1247	0.2123	Gradyan istikrar izleme (moment ataleti)
	10	0.1746	0.2621	Küçük veri setinde temkinli/yavaş ilerleme

6.2. Hiperparametre Isı Haritası (50. Epok Sonu Kaybı)

Tablo 6.3 — Öğrenme Oranı × Optimizatör Duyarlılık Matrisi

Optimizatör	lr = 0.001	lr = 0.01	lr = 0.1
Klasik SGD	19.1728	**0.0321**	0.3093
Momentumlu SGD	2.1045	**0.0012**	**4.8912**
Adam Optimizer	116.8678	85.4459	**0.9736**

6.3. Yorumlama

Küçük, doğrusal ve düşük boyutlu bu örnek veri setinde klasik SGD hızlı ve istikrarlı biçimde yakınsamıştır; momentum bileşeni hıza ivme kazandırırken orta epoklarda aşırı geçiş (overshoot) davranışı üretmiştir. Adam'ın ilk ve ikinci moment tahminlerindeki "ısınma" (warm-up) ataleti, bu ölçekte yavaş görünse de gürültülü/doğrusal-olmayan büyük veri setlerinde daha kararlı bir alternatif olarak literatürde kabul görmektedir; bu nedenle üretim ortamında Procedural Guardian'ın soft-check bileşeninde Adam tercih edilmiştir.

7. PROSEDÜREL UYUM MUHAFIZI — KURAL MOTORU VE KOD EKRANLARI

Hard-check / soft-check hibrit mimarisi

Tablo 7.1 — Kontrol Katmanları

Katman	Mantık	Örnek Kural	Aksiyon
HARD CHECK	İkili (boolean), emredici hüküm	Cevap süresi 2 haftadır (HMK m.127)	İşlem bloke (Hard-Stop)
SOFT CHECK	Olasılıksal, süreç madenciliği	Görevsizlik riski taşıyan dava türü-mahkeme eşleşmesi	Uyarı bayrağı (Soft-Alert)

Kod Ekranı 7.1 — Basitleştirilmiş Kural Motoru Sözde Kodu (Drools tarzı)

```
rule "Tebligat_Suresi_Kontrolu"
when
    $dosya : Dava( turu == "Kira Tespit" )
    $adim  : ProsedurAdimi( ad == "Tebligat", tamamlandi == false,
                            gecenGunSayisi > 14 ) from $dosya.adimlar
then
    guardian.hardStop($dosya, "R01: Tebligat suresi asildi (HMK m.127)");
    auditLog.kaydet(EventType.BLOCK, $dosya.id, "R01");
end
```

Kod Ekranı 7.2 — Uyum Kontrol Algoritması (Python Sözde Kod)

```
def uyum_kontrolu(dava, bugun):
    """Girdi: Dava D, mevcut tarih M / Çıktı: Uyum raporu R"""
    harita = prosedur_haritasi_getir(dava.tur)
    rapor = UyumRaporu(dava_id=dava.id)

    for adim in harita.adimlar:
        tamam = adim.tamamlandi_mi()
        bagimli_tamam = all(b.tamamlandi_mi() for b in adim.bagimliliklar)
        sure_asimi = (bugun - adim.baslangic).gun > adim.azami_sure
        belge_eksik = not adim.gerekli_belgeler_tam_mi()

        risk_skoru = risk_modeli.forward(adim.ozellik_vektoru())  # NumPy çekirdeği

        if sure_asimi and adim.emredici:
            rapor.ekle(Uyari(seviye="HARD", adim=adim, risk=risk_skoru))
        elif risk_skoru > RISK_ESIGI:
            rapor.ekle(Uyari(seviye="SOFT", adim=adim, risk=risk_skoru))

    rapor.genel_skor = rapor.hesapla_toplam_skor()
    return rapor
```

8. ÖRNEK VAKA ANALİZİ — KARMAŞIKLIK MATRİSİ VE METRİKLER

Gösterim amaçlı sentetik senaryo üzerinde değerlendirme

Önemli kapsam notu: Aşağıdaki senaryo, 100 adımlık *kurgusal/örnek* bir sürecin sonuçlarını göstermek için hazırlanmış bir illüstrasyondur; belirli bir gerçek mahkeme veya esas numarasına ait doğrulanmış veri değildir. Gerçek UYAP verisiyle ölçüm, ancak Faz 3 pilot uygulaması sonrasında yapılabilir (bkz. Bölüm 12).

Tablo 8.1 — Karmaşıklık Matrisi (Örnek Senaryo, n = 100 işlem adımı)

	Tahmin: İHLAL VAR	Tahmin: İHLAL YOK	Toplam
Gerçek: İHLAL VAR	**TP = 19**	**FN = 1**	20
Gerçek: İHLAL YOK	**FP = 1**	**TN = 79**	80
Toplam	20	80	100

Tablo 8.2 — Türetilen Performans Metrikleri ve Hedef KPI Karşılaştırması

Metrik	Formül	Hesaplanan	Hedef KPI	Durum
Accuracy	(TP+TN)/Toplam	%98.0	≥ %95.0	✔ **Başarılı**
Precision	TP/(TP+FP)	%95.0	≥ %90.0	✔ **Başarılı**
Recall	TP/(TP+FN)	%95.0	≥ %95.0	✔ **Başarılı**
F1-Skoru	2·P·R/(P+R)	%95.0	≥ %90.0	✔ **Başarılı**
False Positive Rate	FP/(FP+TN)	%1.25	≤ %5.0	✔ **Başarılı**

8.1. Hata Analizi

Tablo 8.3 — Hata Türleri ve İyileştirme Önerileri

Hata Türü	Sayı	Hukukî Sonuç	Teknik İyileştirme
False Positive	1	Gereksiz blokaj → yargılama süresi uzar	Kural motoruna "hâkim onayı ile override" mekanizması
False Negative	1	Usul hatası gözden kaçar → temyizde bozma riski	OCR barkod doğrulama + çapraz referans kontrolü

9. FEDERATED LEARNING VE BLOCKCHAIN DENETİM KATMANI

Veri gizliliği ve değiştirilemez denetim izi

UYAP verisi KVKK m.6 anlamında özel nitelikli kişisel veri kapsamına girebileceğinden, merkezi ham veri toplanması yerine **Federated Learning (Federe Öğrenme)** mimarisi önerilmektedir: her adliye modelini kendi yerel verisiyle eğitir, merkeze yalnızca şifreli gradyan güncellemeleri gönderilir.

Tablo 9.1 — Federated Learning Parametreleri

Parametre	Değer	Gerekçe
Algoritma	FedProx + DP-SGD	Non-IID veri dağılımı + KVKK uyumu
Round sayısı	100 round / hafta	Yakınsama gözlemine göre ayarlanır
Diferansiyel gizlilik (ε)	0.1	Güçlü gizlilik / doğruluk dengesi
Secure Aggregation	Thales HSM + TLS 1.3	KVKK + ISO 27001 uyumu
Framework	PySyft / Flower	Açık kaynak, on-prem dağıtıma uygun

Kod Ekranı 9.1 — Denetim Kaydı Şeması (Hash Zinciri)

```
record = {
    "event_type": "BLOCK" | "ALERT" | "OVERRIDE",
    "case_id": "<dosya-no>",
    "rule_id": "R01",              # örn. Tebligatsız celse
    "user_id": "<hakim-kullanici-id>",
    "override_justification": "...",  # override ise zorunlu
    "timestamp": "2026-05-15T10:23:45Z",
    "previous_hash": "a3f1...9c2d",
    "hash": SHA256(record)          # değiştirilemez
}
```

İkinci katmanda, her günün log Merkle kökü izin-denetimli (permissioned) bir Hyperledger Fabric ağına periyodik olarak "anhorlanır"; bu sayede bağımsız denetim mercileri (HSK, BAM) kayıtların bütünlüğünü kriptografik olarak doğrulayabilir. Public blok zincirler (Bitcoin/Ethereum) KVKK'nın veri yerelliği ilkesiyle bağdaşmadığından tercih edilmemiştir.

10. API TASARIMI VE SİSTEM ENTEGRASYONU

RESTful uç noktalar ve UYAP 2.0 senkronizasyonu

Tablo 10.1 — Ana API Uç Noktaları

Metod	Uç Nokta	Açıklama
POST	/api/v1/davalar	Yeni dava oluştur
GET	/api/v1/davalar/{id}	Dava detayı
GET	/api/v1/davalar/{id}/uyum	Uyum raporu
POST	/api/v1/davalar/{id}/adimlar	Prosedür adımını tamamla
GET	/api/v1/prosedurler	Prosedür haritası listesi
POST	/api/v1/uyap/senkronizasyon	UYAP ile çift yönlü senkronizasyon
GET	/api/v1/uyap/durum	UYAP bağlantı durumu

Kod Ekranı 10.1 — FastAPI Uç Nokta Taslağı

```python
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI(title="Procedural Guardian API")

class UyumRaporuYaniti(BaseModel):
    dava_id: str
    genel_skor: float
    uyarilar: list

@app.get("/api/v1/davalar/{dava_id}/uyum", response_model=UyumRaporuYaniti)
def uyum_raporu_getir(dava_id: str):
    dava = dava_servisi.getir(dava_id)
    if not dava:
        raise HTTPException(status_code=404, detail="Dava bulunamadi")
    rapor = uyum_kontrolu(dava, bugun=simdi())
    return rapor
```

11. ETİK, HUKUKİ UYUM VE RİSK YÖNETİMİ

CEPEJ, AB AI Act ve KVKK çerçevesinde tasarım ilkeleri

Tablo 11.1 — Risk Kayıt Defteri (Risk Register)

Risk	Olasılık	Azaltım Stratejisi
Algoritmik önyargı	Orta-Yüksek	Veri seti dengeleme, periyodik disparate impact analizi, model kartları
Halüsinasyon (LLM/RAG)	Düşük-Orta	Üretim yalnızca retrieval'a dayanır; kaynak doğrulama; insan onayı
Yargı bağımsızlığı ihlali	Düşük	AI yalnızca usul denetimi yapar; esas karara karışmaz; override her zaman hâkimde
KVKK / veri sızıntısı	Düşük	On-prem, AES-256, HSM, differential privacy, yıllık sızma testi

11.1. Tasarım İlkeleri

- **İnsan-Merkezlilik:** Sistem yalnızca destek sağlar; nihai takdir hâkime aittir.
- **Açıklanabilirlik (XAI):** Her öneri, dayandığı emsal/kural referansıyla birlikte sunulur.
- **Veri Minimizasyonu:** Kişisel tanımlayıcılar (T.C. kimlik no, ad-soyad) pseudonimleştirilir.
- **Denetlenebilirlik:** Audit trail, SHA-256 hash zinciri ve periyodik blok zincir anhorlaması ile korunur.

12. MALİYET-GELİR ANALİZİ VE YOL HARİTASI

36 aylık finansal projeksiyon (planlama senaryosu)

Not: Aşağıdaki tutarlar, olası bir ticarileşme senaryosu için hazırlanmış *planlama varsayımlarıdır*; gerçekleşmiş gelir/gider verisi değildir.

Tablo 12.1 — Yıllık Maliyet Kalemleri (₺)

Kalem	Yıl 1	Yıl 2	Yıl 3
AI/ML Mühendis Ekibi	3.200.000	3.500.000	4.000.000
GPU Altyapı (on-prem)	4.500.000	1.200.000	1.200.000
Veri Seti / Anonimleştirme	1.200.000	400.000	200.000
Yazılım ve Güvenlik	800.000	600.000	600.000
Toplam	**10.700.000**	**7.100.000**	**7.300.000**

Tablo 12.2 — Kilometre Taşları

Ay	Kilometre Taşı
3	Veri seti hazır, KVKK anonimleştirme ve etik kurul onayı tamamlandı
7	HukukBERT v1.0 eğitildi; NER F1 ≥ 0.84
12	RAG hattı canlı, Procedural Guardian kural motoru aktif
18	İstanbul + Ankara pilot tamamlandı, 50 hâkim değerlendirmesi
26	Break-even hedefi (planlama senaryosu)
36	250 adliye entegrasyonu hedefi, MENA genişleme hazırlığı

13. SONUÇ VE GELECEK ÇALIŞMALAR

Bulgular, sınırlılıklar ve öneriler

13.1. Elde Edilen Bulgular

- Hazır çerçeve kullanılmadan, tamamen NumPy ile ileri/geri yayılım ve üç optimizatör sıfırdan kodlanmıştır.
- Kavramsal olarak hibrit (kural tabanlı + istatistiksel) bir Prosedürel Uyum Muhafızı mimarisi tasarlanmıştır.
- KVKK uyumlu bir federated learning ve blok zincir tabanlı denetim izi önerisi geliştirilmiştir.

13.2. Sınırlılıklar

- Bölüm 6 ve 8'deki sayısal sonuçlar küçük ölçekli, sentetik/örnek verilerle üretilmiştir; gerçek UYAP verisiyle doğrulama yapılmamıştır.
- UYAP 2.0 resmi API'si şu an tam açık değildir; entegrasyon simülasyon ortamında test edilmiştir.
- NLP modeli, sınırlı hacimli Türkçe hukuki veriyle prototiplenmiştir; kapsamlı fine-tuning gelecek çalışma konusudur.

13.3. Gelecek Çalışmalar

- Anonimleştirilmiş gerçek UYAP loglarıyla "gölge modda" (shadow mode) pilot değerlendirme.
- Çok dilli destek ve MENA/Orta Asya bölgesine genişleme fizibilitesi.
- Bağımsız üçüncü taraf denetimiyle bias audit ve XAI raporlarının kurumsallaştırılması.

KAYNAKÇA

Aletras, N., Tsarapatsanis, D., Preoţiuc-Pietro, D., & Lampos, V. (2016). Predicting judicial decisions of the European Court of Human Rights: a natural language processing perspective. *PeerJ Computer Science, 2*, e93.

Chalkidis, I., Fergadiotis, M., Malakasiotis, P., Aletras, N., & Androutsopoulos, I. (2019). Extreme multi-label legal text classification: A case study in EU legislation. *ACL*.

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design Science in Information Systems Research. *MIS Quarterly, 28*(1), 75–105.

Katz, D. M., & Bommarito, M. J. (2014). Measuring the complexity of the law: The United States Code. *Artificial Intelligence and Law, 22*(4).

Papagianneas, S. (2024). *The Digitalization of the Chinese Judiciary: A Smart Court Approach*. Springer.

Susskind, R. (2019). *Online Courts and the Future of Justice*. Oxford University Press.

CEPEJ. (2018). *European Ethical Charter on the Use of Artificial Intelligence in Judicial Systems*. Council of Europe.

European Union. (2024). *Artificial Intelligence Act*. European Parliament.

Adalet Bakanlığı. (2025). *Yargı İstatistikleri*. https://www.adalet.gov.tr/

Hukuk Muhakemeleri Kanunu (HMK), Kanun No: 6100.

Türk Borçlar Kanunu (TBK), Kanun No: 6098.